

ARGLY

· Antifraude de Siniestros

Detector de posible fraude en siniestros, explicable y agentic.

hackIathon 2026 · Reto Aseguradora del Sur

⚠ Argly genera **alertas de revisión, no acusaciones** · la decisión final es del analista humano

0.93

AUC HÍBRIDO

0.97

PRECISIÓN

1392

SINIESTROS

\$1.6M

EN REVISIÓN

66

TESTS VERDES

Problema Solución Arquitectura Score Demo Métricas Pruebas de fuego Rúbrica
Ética Cómo correrlo

1 · El problema

- La detección manual depende del ojo del analista, reglas dispersas y cruces lentos.
- Un siniestro suelto puede verse limpio: el fraude **organizado** vive en las relaciones (anillos de proveedores, narrativas clonadas, frecuencias atípicas).

- Resultado: tiempo perdido revisando casos limpios y exposición sostenida en pagos sospechosos.

2 · La solución — Argly

Plataforma web interna que **prioriza reclamos** con un score **0-100 explicable** + semáforo    · sistema **híbrido** de reglas + ML + anomalías + grafo + NLP + agente de IA.

Qué hace

- Genera score y razón por cada siniestro.
- Detecta **anillos** de proveedores (grafo).
- Aísla **narrativas clonadas** (NLP).
- Responde en lenguaje natural (agente IA).
- Emite PDF de investigación.

Qué NO hace

- NO acusa de fraude.
- NO rechaza siniestros ni decide pagos.
- NO usa datos reales: 100% sintéticos para el reto.
- NO reemplaza al analista; lo prioriza y le explica.

3 · Cómo funciona

- **Score por contribuciones transparentes:** cada motor devuelve {señal, puntos, evidencia} ; la fusión las suma a 0-100 + semáforo. El **desglose ES la suma**.
- **Hard gates** (RF-02 adulteración documental, RF-03 lista restrictiva, RF-04 dinámica inconsistente) fuerzan **ROJO** con precedencia.
- **Agente IA** planifica acciones y llama herramientas **deterministas** (catálogo de tools): no inventa números, los obtiene de la bandeja real.

4 · Cómo se construye el score (ejemplo)

Siniestro tipo "robo a 24h de haber contratado la póliza, reportado 6 días después, monto 97% de la suma asegurada":

+8 Borde de vigencia

Siniestro a 1 días del inicio de la póliza (≤ 10)

+8 Demora denuncia robo

Robo reportado 6 días después ($> 48h$)

+5 Monto cercano a suma asegurada

Reclamo = 97% de la suma asegurada

+48 Modelo ML

Probabilidad de fraude del modelo: 95%

Total: **69** · **AMARILLO** · **Recomendación:** escalar para revisión documental.

5 · Demo en vivo (4 minutos)

- 1 **Bandeja "Most Wanted"** priorizada por riesgo, con filtro por semáforo y monto en revisión visible.
- 2 **Detalle del caso:** score, desglose por regla, evidencia, recomendación y botón **Descargar dossier PDF**.
- 3 ☐ **Scorear un siniestro nuevo:** el jurado define los inputs (o usa el preset "robo a 24h") y ve el score + el porqué **en vivo**.
- 4 ☐ **Pregúntale a Argly** en lenguaje natural: "*¿qué proveedores concentran las alertas rojas?*" → responde con los anillos detectados (P0001 / P0002).
- 5 ☐ **Redes detectadas:** cards con proveedor, n_siniestros, n_asegurados y exposición.

6 · Métricas reales (test held-out, 422 casos)

Métrica	Valor
AUC solo-reglas	0.85
AUC híbrido	0.93
AUC modelo ML	0.95
Precisión	0.97
Recall	0.80
F1	0.88

Distribución	Casos
VERDE flujo normal	1146
AMARILLO revisión documental	172
ROJO revisión especializada	74
Monto en revisión	\$1.605.659
Total siniestros	1392
Tests automatizados	66 ✓

Sistema reproducible (semilla fija). Train/test separados: los anillos del test usan proveedores **distintos** a los del entrenamiento (sin métricas circulares).

7 · Pruebas de fuego del jurado

Prueba (sec. 23 del reto)	Cubierta por	Estado
Consulta agéntica en lenguaje natural	<code>POST /api/preguntar</code> + chat en la web	✓
Scorear un siniestro nuevo en vivo y explicar el riesgo	<code>POST /api/scorear</code> + form en la web	✓
Verificación del repositorio modular en GitHub	<code>github.com/SebasB76/Argly</code> · 66 tests · <code>run.sh</code>	✓

8 · Cobertura de la rúbrica

Criterio	Peso	Cubierto por
Uso de IA y prototipo	40%	ML + NLP + anomalías + agente NL con tool-calling
Explicabilidad y ética	25%	Contribuciones transparentes + gates + sello + trazabilidad
Análisis del caso	15%	Grafo de redes (networkx) + cruce multivariable
Tecnología y arquitectura	10%	Repo modular + API REST + 66 tests + run.sh + Mermaid docs
Pitch e impacto	10%	Demo redonda + métricas + monto + roadmap

9 · Ética, trazabilidad y datos

- **Score descomponible** hasta el dato origen (cada regla, sus puntos, su evidencia).
- **Sello "alerta ≠ acusación"** en API, web, PDF y avisos.
- **Humano en el ciclo:** la API no decide pagos.
- **Datos 100% sintéticos**, sin PII real, sin credenciales en el repo (`.env` ignorado).
- **Train/test honesto:** los rings del test usan proveedores distintos a los del train.
- **Auditoría de sesgo** por ciudad/segmento como práctica.

10 · Cómo correrlo

```
cd argly
./run.sh setup      # crea venv, instala deps de Python y del front
./run.sh demo       # levanta API (:8000) + web (:5173)
# abre http://localhost:5173
```

Otros comandos: `./run.sh test` (66 tests) · `./run.sh pipeline` (bandeja + métricas) · `./run.sh api` (Swagger en `/docs`).

Endpoints clave

Endpoint	Devuelve
<code>GET /api/resumen</code>	totales por semáforo + monto en revisión
<code>GET /api/casos?nivel=&limit=</code>	bandeja priorizada
<code>GET /api/casos/{id}</code>	score + desglose + evidencia + recomendación
<code>POST /api/scorear</code>	puntúa un siniestro nuevo en vivo + explica
<code>POST /api/preguntar</code>	agente: respuesta en lenguaje natural
<code>GET /api/redes</code>	anillos (proveedores que concentran alertas)
<code>GET /api/narrativas-similares</code>	pares de narrativas casi idénticas
<code>GET /api/casos/{id}/dossier</code>	PDF de investigación
<code>GET /api/casos/{id}/aviso</code>	preview WhatsApp/correo
<code>POST /api/casos/{id}/push</code>	push del score al core (mock)

11 · Stack y próximos pasos

Stack: Python · FastAPI · PostgreSQL/SQLite · React (Vite) · scikit-learn · networkx · sentence-similarity (TF-IDF/coseno) · LLM compatible-OpenAI (Gemini / DeepSeek / Groq).

Próximos pasos

- **Integración real:** WhatsApp/correo con credenciales del cliente; push al core (Oracle).
- **Refinamiento:** re-entrenar con datos reales bajo gobernanza; calibración por ramo.
- **Operación:** dashboards de tendencias y alertas en tiempo real.

- **Gobierno del modelo:** auditoría periódica de sesgo, política de retraining.

Repo: github.com/SebasB76/Argly · [./run.sh demo](#) · 66 tests verdes · hackIAthon 2026.